

Глубинное обучение в обработке звука

*Speaker Diarization и
Source Separation*

План лекции

- *Voice Activity Detection*

- *Speaker Diarization*

- *Source Separation*

Voice Activity Detection

Voice Activity Detection

Цель:

- Хотим отделить полезный сигнал от всего остального



Будем решать задачу бинарной классификации:

- Речь – класс 1
- Тишина\шум – класс 2

Применение VAD

Экономия данных: не передавать тишину по сети

ASR: Снижение нагрузки на сервер

Keyword Spotting: Экономия заряда батареи

Очистка данных: Удаления тишины для обучения *ASR/TTS* моделей

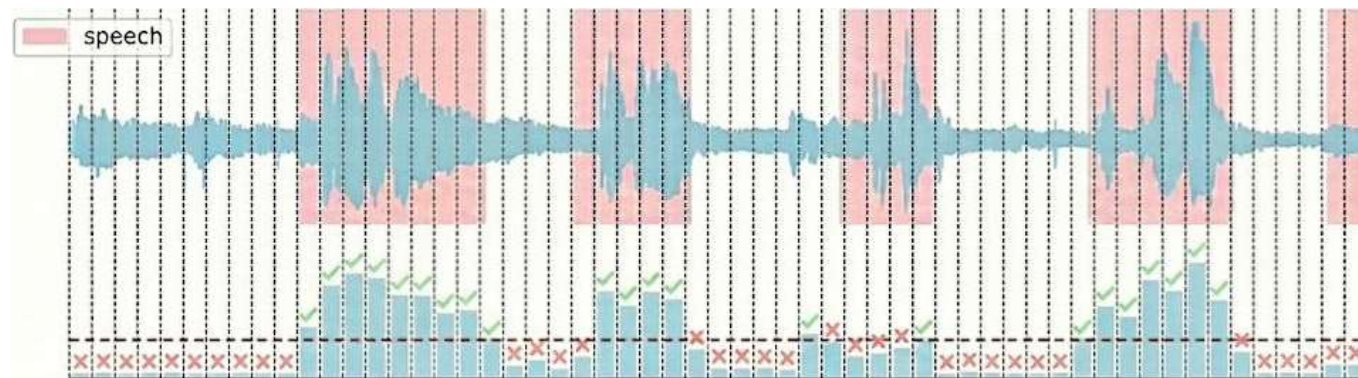
Voice Activity Detection

Идея:

Речь обычно громче фона, поэтому можно отделять их по некоторому порогу

Алгоритм:

- Разбиваем сигнал на фреймы
- Считаем энергию для каждого фрейма
- Сравниваем эти значения с порогом



Voice Activity Detection

Минусы:

- Фоновые шумы иногда могут быть громче речи
- При низком SNR сложно подобрать порог
- Окончания фраз и глухие согласные часто имеют низкую энергию

Метрики VAD

Аудио бьётся на фреймы и для каждого фрейма считаются:

- **TP** – речь была и модель нашла её
- **TN** – была тишина и модель нашла ее
- **FP (False Alarm)** – была тишина, модель предсказала речь
- **FN (Missed Detection)** – речь была, модель предсказала тишину

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Метрики VAD

Основные метрики:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

- *Precision* показывает насколько мы доверяем положительному срабатыванию
 - Низкий *Precision* → в запись попадает много шумов, дыхания
-
- *Recall* показывает какую долю реальной речи мы нашли
 - Низкий *Recall* → теряются начало\концы фраз и тихие слова

Метрики VAD

Detection Error Rate (DER):

$$DER = \frac{False\ Alarm\ Time + Missed\ Detection\ Time}{Total\ Speech\ Time}$$

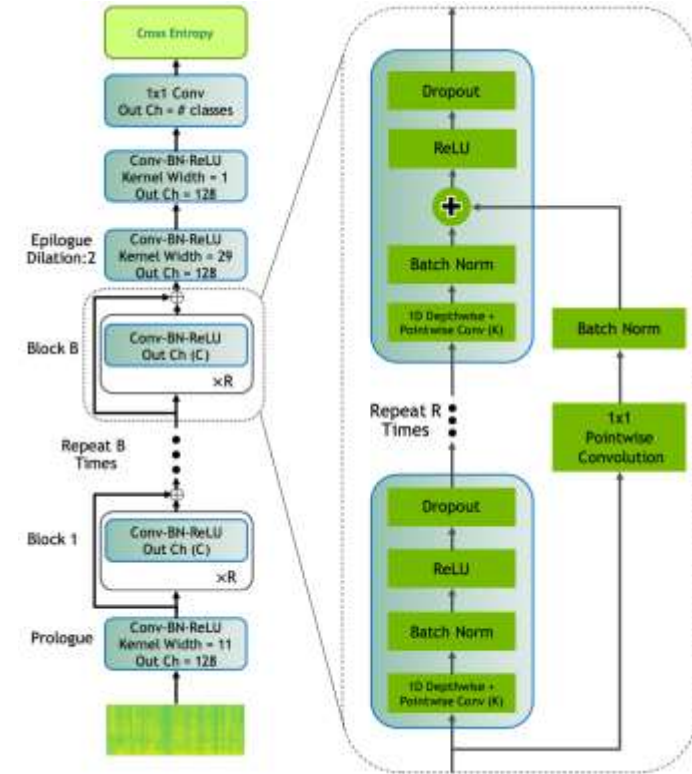
- *False Alarm Time* — сколько шума пропустила система
- *Missed Detection Time* — доля потерянного сигнала

Silero VAD

- На вход подается спектрограмма
- Кодировщик – неглубокая свёрточная нейросеть
- Контекст обрабатывается с помощью *LSTM*
- Голова – полносвязный слой

MarbleNet

- По сути, обрезанный *QuartzNet*
- В основе лежит *Jasper Block*
- Состоит из 1D свёрток



Pyannote VAD

Используются *SincNet* и *Temporal Convolution Network*:

- Первый слой инициализируется функцией $\text{sinc}(x)$
- Сеть учит 2 параметра – границы фильтров f_{H} и f_{V}
- Это заставляет сеть смотреть на определенные частотные диапазоны
- *TCN* состоит из *Dilated Convolutions*

Another VAD models

Идея 1:

- Взять предобученную *self – supervised* модель (*Wav2Vec2/HuBERT*) и обучить свой *VAD* классификатор

Идея 2:

- Использовать *ASR CTC* модель для *VAD*

Модели VAD

Модель	Тип	Latency / RTF	Точность (Quality)	Лучший кейс
Silero VAD	Hybrid / Compact	Ultra Low	Good	Real-time streaming, голосовые боты, edge-устройства.
WebRTC VAD	GMM	Instant	Low/Mid	Когда CPU критичен и шум низкий.
SpeechBrain	CRNN / Hybrid	Mid	High	Кастомные пайплайны, научные задачи, файн-тюнинг.
MarbleNet	CNN	Mid/High	High	Задачи, где важна устойчивость к шуму, но не критичен стриминг <50ms.
Pyannote 3.x	Segmentation	Offline / Slow	SOTA	Диаризация, пост-анализ записей, сложные акустические сцены.
Whisper	Transformer (ASR)	High	Semantic SOTA	Сегментация длинных видео, субтитры, где точность > скорости.

Speaker Diarization

Speaker Diarization

Хотим ответить на вопрос: *кто и когда говорил?*

- Это задача обучения без учителя (*unsupervised learning*), поскольку нам не нужны метки
- В результате получаем временную разметку по спикерам



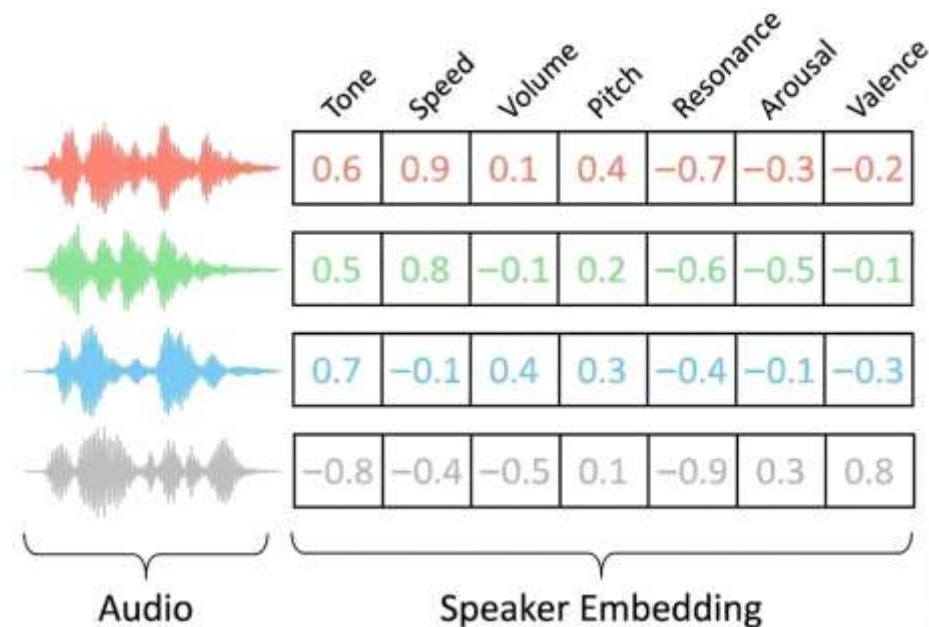
Diarization Pipeline: Обработка сигнала

Сегментация и VAD:

- Запись разделяется на короткие отрезки по несколько секунд обычно с перекрытием
- VAD отсеивает тишину и шум, оставляя полезный сигнал

Создание эмбеддингов:

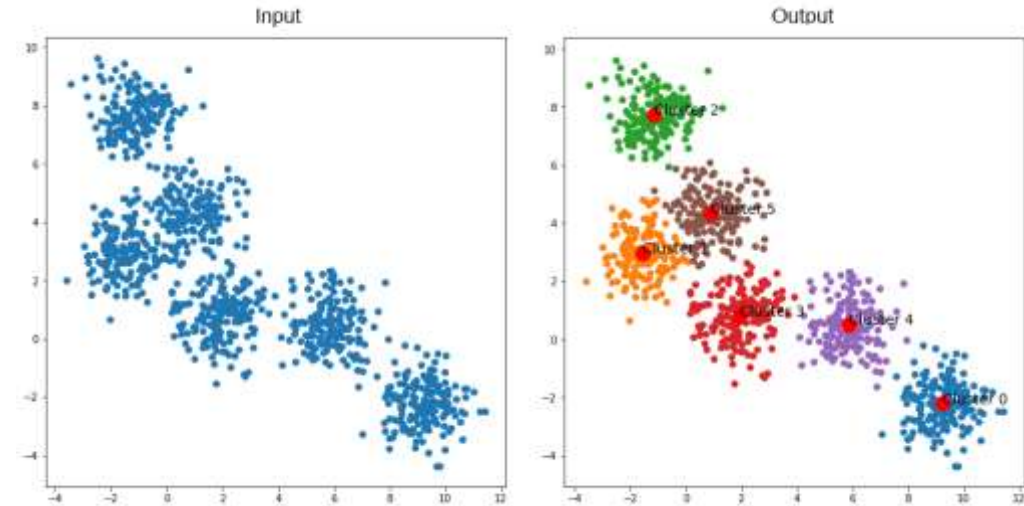
- С помощи модели кодировщика получаем эмбеддинги спикера
- Эти вектора содержат основные характеристики голоса



Diarization Pipeline: Кластеризация

Кластеризация:

- С помощью алгоритмов кластеризации похожие векторы группируются в кластеры
- Сегментам одного кластера присваивается единая метка *speaker1, speaker2, ...*



Метрики качества

Diarization Error Rate (DER) – измеряет долю времени записи, размеченной неверно:

$$DER = \frac{False\ Alarm + Missed\ Speech + Speaker\ Confusion}{Total\ Speech\ Time}$$

Иногда используется ***Jaccard Error Rate (JER)*** – оценивает каждого спикера равнозначно, независимо от того, как много он говорил

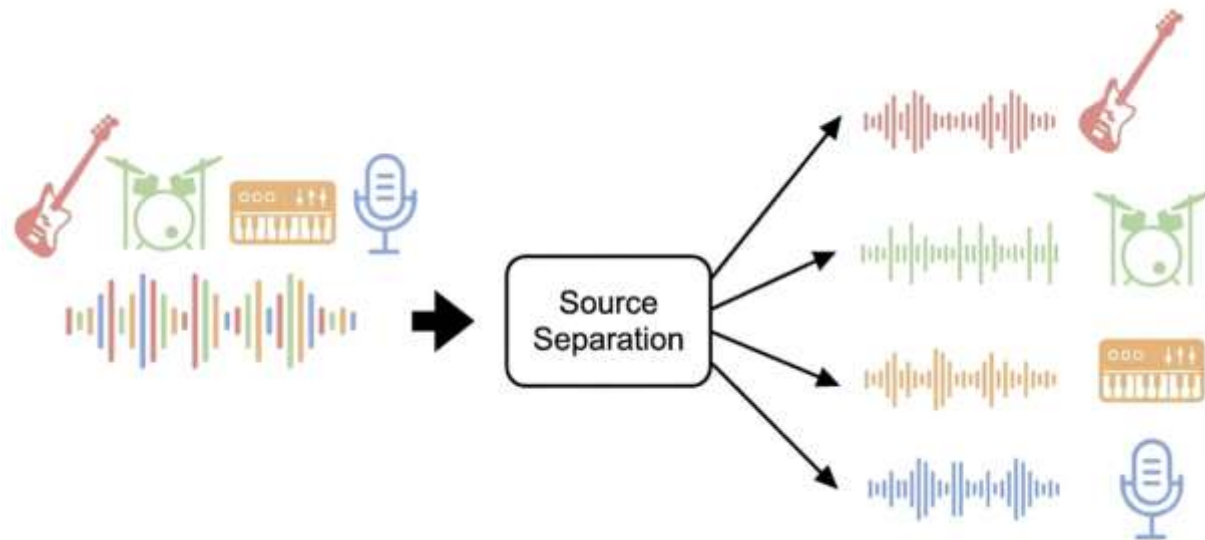
Source Separation

Source Separation

Нужно выделить конкретный голос или инструмент из смеси других звуков

Применение:

- Удаление шума
- Получение дорожек из музыкального трека
- Улучшение слуховых аппаратов
- Диаризация спикеров



Метрики *Source Separation*

Perceptual Evaluation of Speech Quality (PESQ) – алгоритм, моделирующий человеческое ухо:

- Сигналы выравниваются по времени
- Считается спектр сигналов
- Частоты переводятся в шкалу Барков, амплитуды – в громкость, учитывая пороги слышимости
- Сравниваются спектрограммы эталона и предсказания
- На основе ошибок вычисляется оценка от -0.5 до 4.5

Метрики *Source Separation*

Signal to Distortion Ratio (SDR):

$$SDR = 10 \log_{10} \left(\frac{\|s_{true}\|^2}{\|e_{interf} + e_{noise} + e_{artif}\|^2} \right)$$

s_{true} — целевой сигнал

e_{interf} — компоненты других источников

e_{noise} — исходный фоновый шум

e_{artif} — остатки (галлюцинации нейросети, бульканье)

Обычно SDR считают по упрощенной формуле, совпадающей с ***Signal Noise Ratio (SNR)***:

$$SDR = SNR = 10 \log_{10} \frac{\|y\|^2}{\|\hat{y} - y\|^2}$$

- $\|y\|^2$ — энергия чисто сигнала
- $\|\hat{y} - y\|^2$ — энергия ошибки

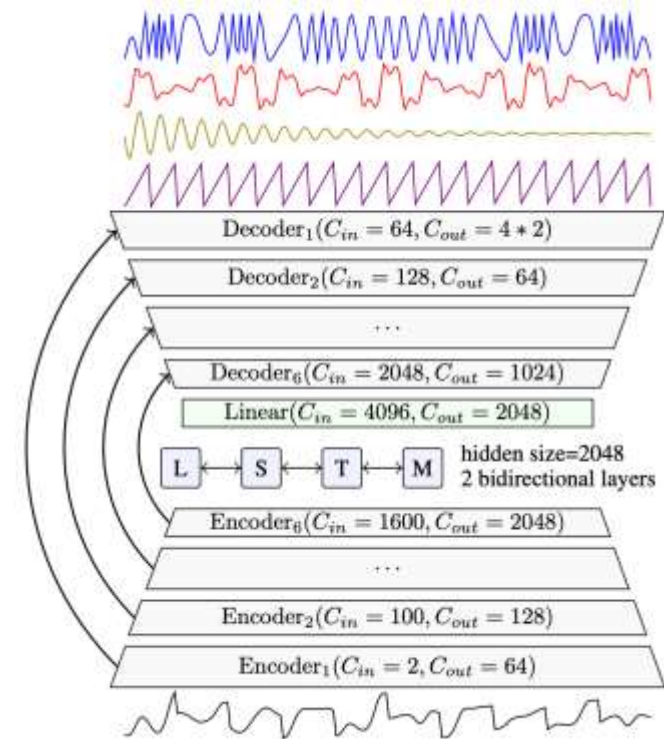
Demucs, 2019-2020

Идея:

- Будем напрямую работать с сигналом, без спектрограмм
- Это позволит избежать проблем с восстановлением фазы

Deep Music Separation (Demucs):

- Используется *U – Net* архитектура
- Энкодер сжимает аудио, увеличивая количество каналов
- *BiLSTM* анализирует сжатое представление песни
- Декодер делает *upsampling* и восстанавливает сигнал
- В *Demucs v2* добавили аугментаций и дополнительное слагаемое в лосс



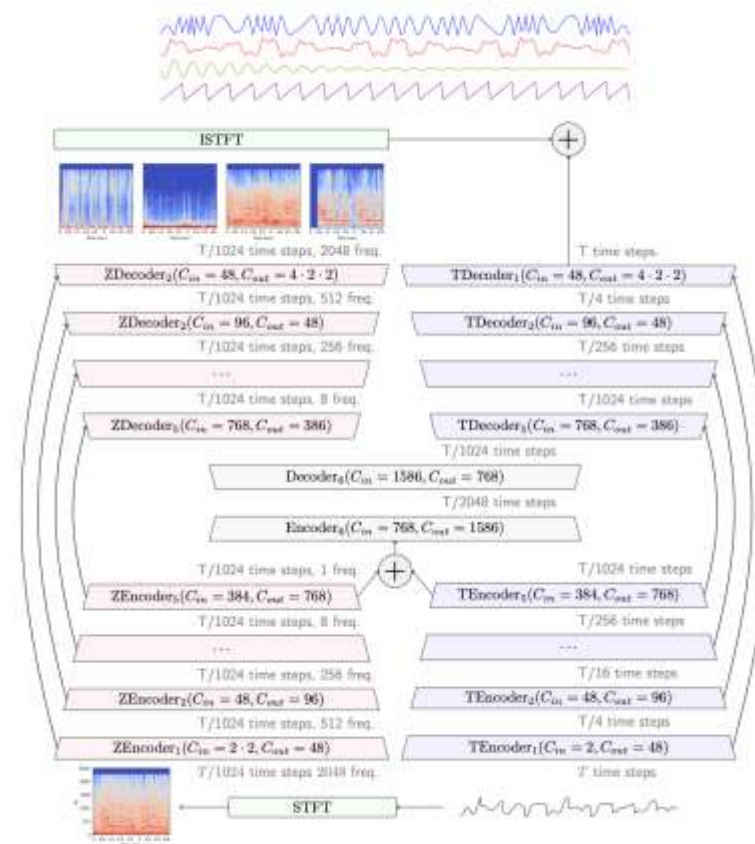
Hybrid Demucs, 2021

- Модели, напрямую работающие с сигналом часто имеют артефакты на высоких частотах

Hybrid Demucs:

- Входной сигнал делится на 2 параллельные ветки:

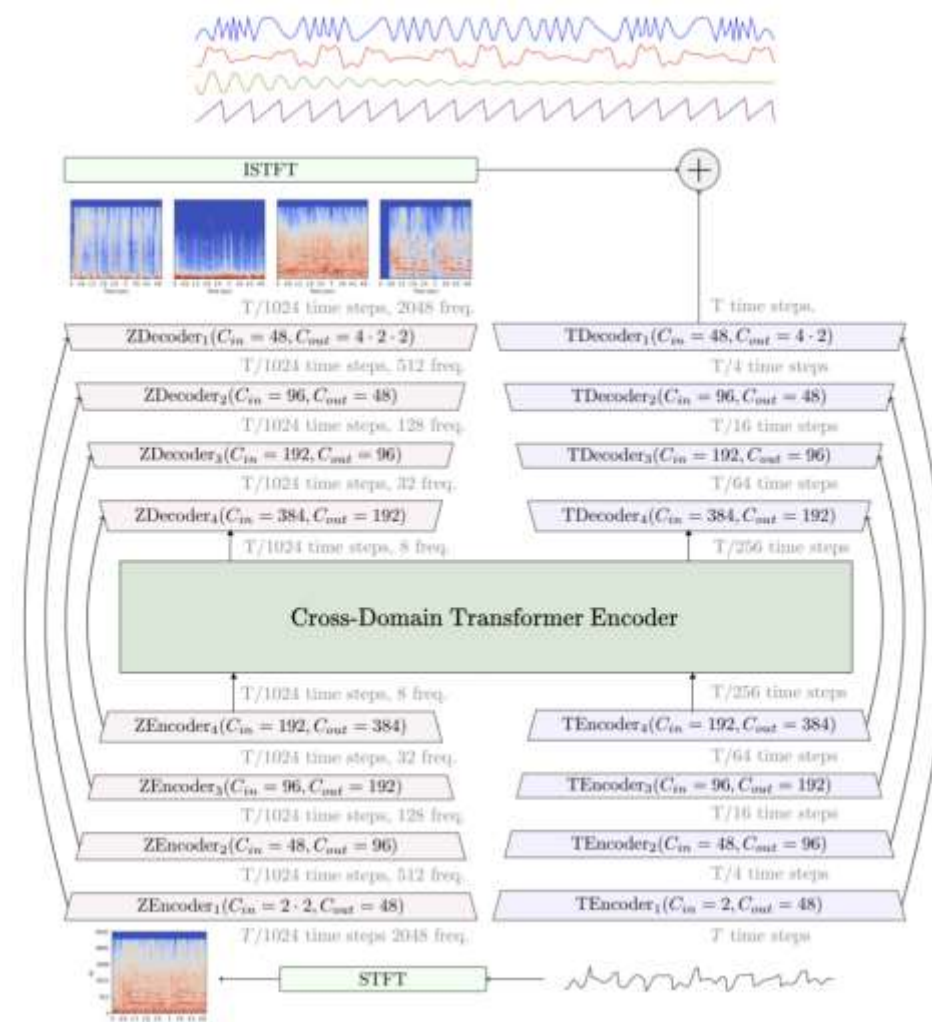
Temporal Branch
Spectral Branch



Hybrid Transformer Demucs, 2022

Hybrid Transformer Demucs:

- Рекуррентные слои заменили на *Cross – Domain Transformer*
- Трансформер берёт данные из спектральной и временной веток и обрабатывает их с помощью механизма внимания



Результаты HT Demucs



Mixture



Drums



Bass



Vocal



Other

Band – Split RNN, 2022

Проблемы прошлых моделей:

- Были взяты из обработки речи или изображений и не учитывали особенностей музыки
- Музыкальные сигналы имеют более высокую частоту дискретизации
- Спектр обрабатывался либо целиком, либо грубо делился на высокие или низкие частоты

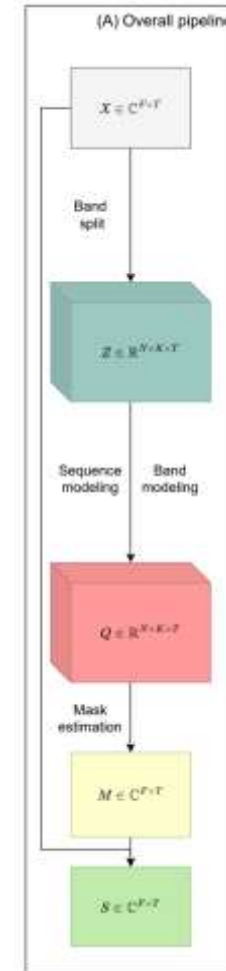
Идея BS RNN:

- Будем явно делить спектрограмму на частотные подполосы (*subbands*) с настраиваемой шириной
- Это вносит априорные знания в модель о природе инструментов

Общая архитектура BS RNN

Основные модули:

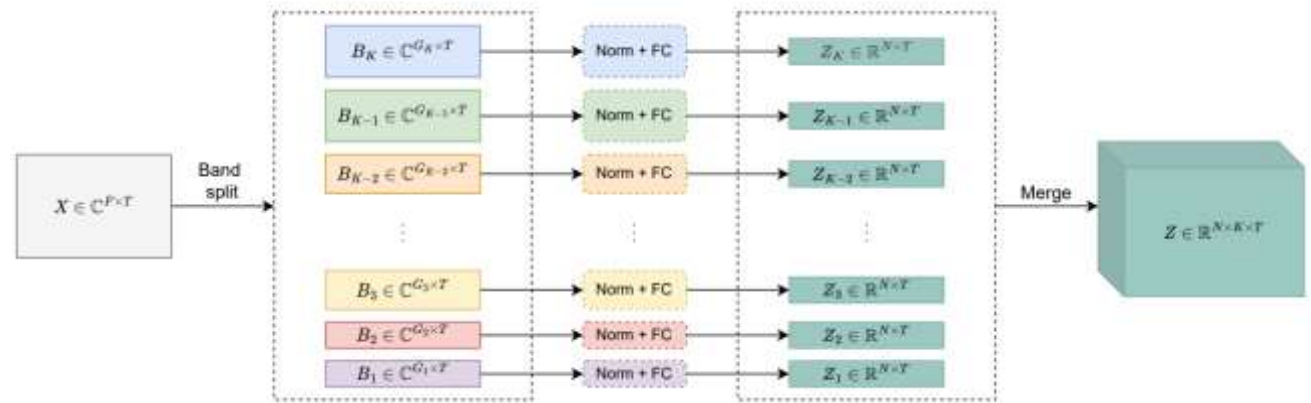
- **Band – Split Module** разделяет входную комплексную спектрограмму на K подполос
- **Band and Sequence Modeling** моделирует временные последовательности и связи между частотными полосами
- **Mask Estimation** создает комплекснозначные маски для каждой полосы



Band – Split Module

Band – Split Module:

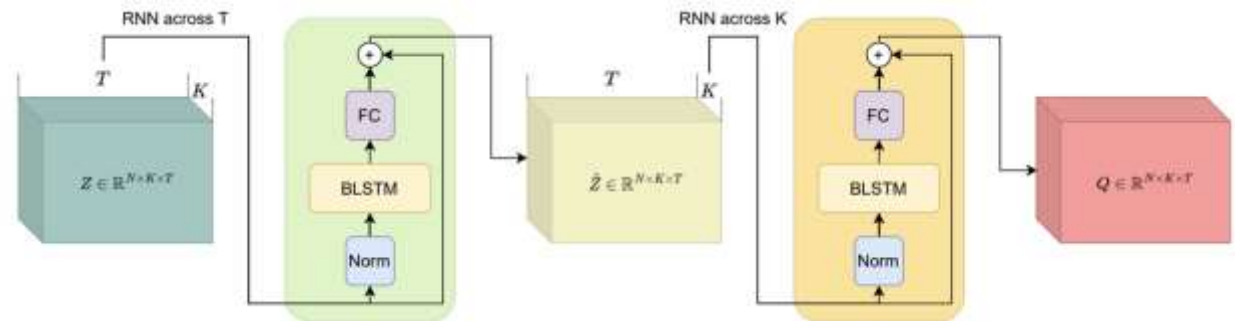
- Вход X нарезается на K подполос B_i
- Ширина полос G_i не фиксирована, а задается на основе знаний об инструменте
- Каждая подполоса B_i проходит через нормализацию и полносвязную сеть FC
- Все полосы объединяются в общий тензор Z



Band and Sequence Modeling

Идея Dual – Path RNN:

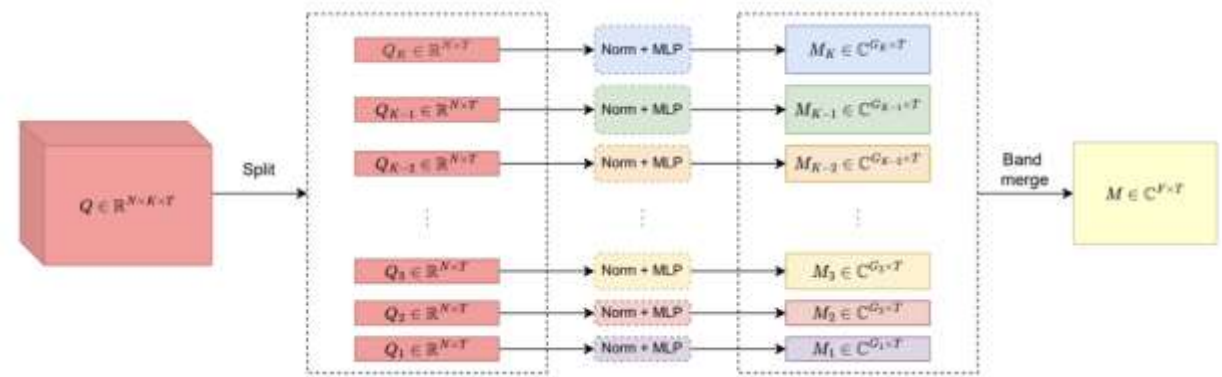
- Модель учится понимать контекст и по времени, и по частоте
- **Sequence Level RNN** обрабатывает каждую полосу независимо, отвечает за понимание временной структуры
- **Band Level RNN** обрабатывает каждый временной шаг, смотря на все полосы сразу, моделирует зависимости между полосами



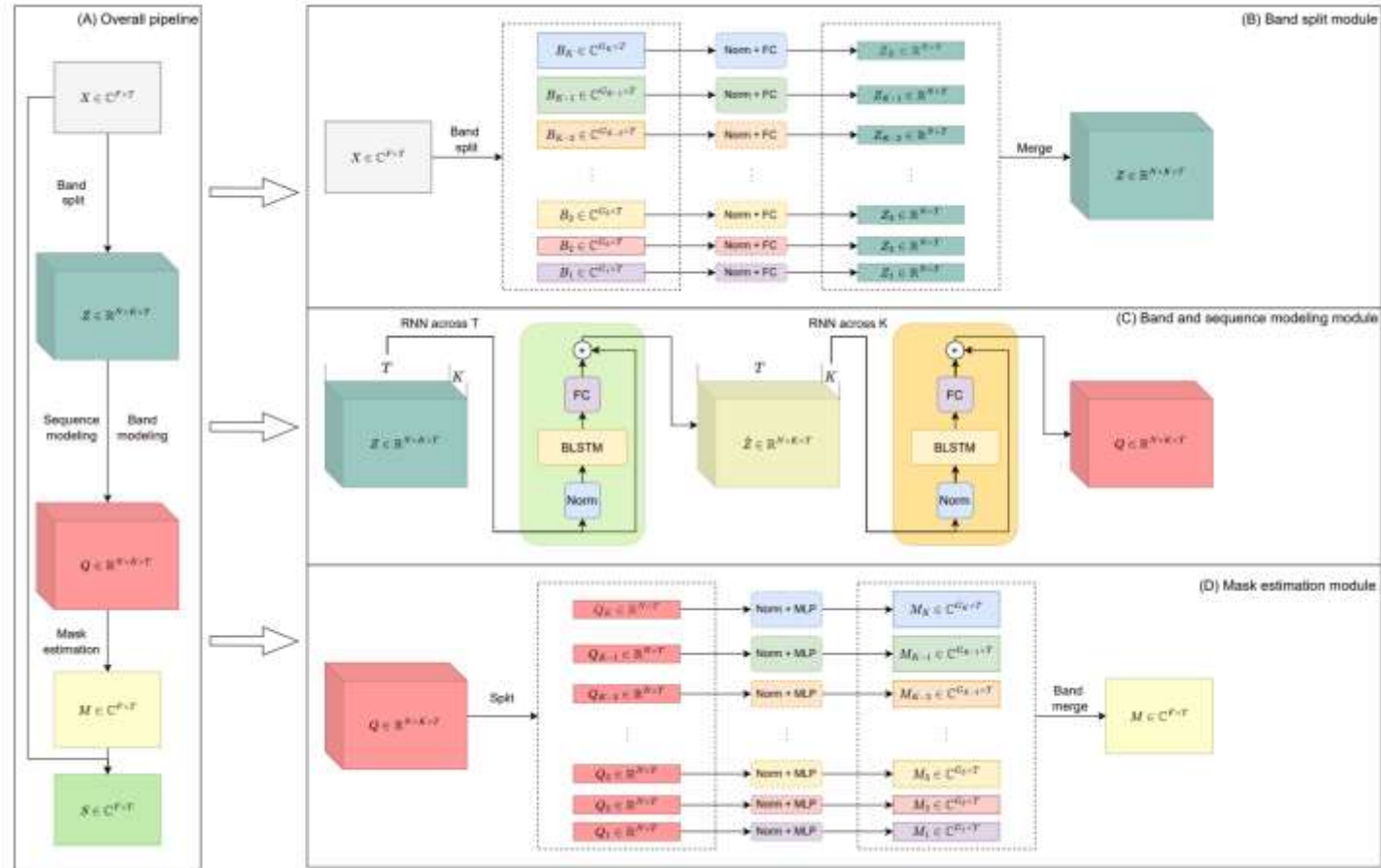
Mask Estimation Module

Mask Estimation Module:

- Делим выход прошлого модуля на K потоков
- Каждый поток проходит через слои нормализации и MLP
- Получаем комплексные маски M_i , которые затем объединяются в общую маску M
- Маска M умножается на исходную спектрограмму X и применяется $iSTFT$ для получения аудио



Band – Split RNN



BS RoFormer, 2023

Проблемы BS – RNN:

- *CNN* и *RNN* ограничены в обработке длинных последовательностей

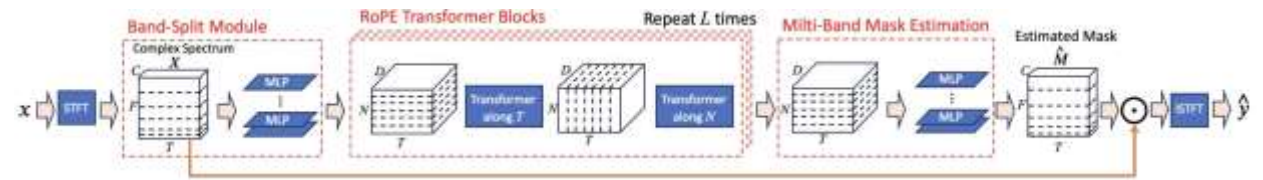
Идея BS RoFormer:

- Будем использовать разделение на частотные полосы и иерархический трансформер с *RoPE*

Общая архитектура *BS RoFormer*

Основные модули:

- ***Band – Split Module*** делит спектрограммы на подполосы
- ***Rope Transformer Blocks*** работает и с временными, и с частотными полосами
- ***Mask Estimation*** создает комплекснозначные маски для каждой полосы



Спасибо за внимание!